

Programmation Python (fonctions, instructions conditionnelles, boucles, bibliothèques)

Programmation Python - les bases

1. Introduction



Dans un langage de programmation, on utilise ce qu'on appelle des **fonctions**. Une fonction est un ensemble d'instructions qui peut recevoir des **paramètres** (qui sont des *valeurs* ou des *variables*) et qui peut **renvoyer le contenu d'une ou plusieurs variables**.

Syntaxe d'une fonction Python

Une fonction Python a la syntaxe suivante :

```
def nom_fonction(parametre1, parametre2, etc.):  
    instruction1  
    instruction2  
    ...  
    return valeur_renvoyee_1, valeur_renvoyee_2, etc.
```

Analyse :

- Le mot clé **def** annonce que l'on va écrire une fonction ;
- On commence par écrire le **nom** de notre fonction ;
- Ensuite, entre parenthèses, on écrit ses **paramètres** séparés par des virgules (il se peut qu'une fonction ne possède pas de paramètres, auquel cas on écrit tout de même les parenthèses mais avec rien entre elles) ;
- On termine la ligne par un deux points (**:**) ;
- On **indente** et écrit toutes les **instructions** devant être exécutées par la fonction : toutes les lignes indentées correspondent au *bloc d'instructions* ;
- Enfin, le bloc d'instruction se termine généralement par le mot clé **return** qui permet de **renvoyer** le contenu d'une ou plusieurs variables (séparées par des virgules s'il y en a plusieurs)



Exercice 1 : code puzzle d'une fonction



Question 1 : Voici le code puzzle d'une fonction qui prend en paramètre un nombre d'octets, le convertit en bits, l'affiche et le retourne. Remets les lignes de code dans l'ordre et indente les lignes correctement. Puis exécute le code pour voir si tu as réussi.

```
convertit_en_bits(2)  
return conversion  
print("conversion de ", nombre_octets, " octet(s) en bit(s) :  
", conversion)  
def convertit_en_bits(nombre_octets):  
    # 1 octet = 8 bits  
    conversion = nombre_octets * 8
```

```
Entrée[1]: convertit_en_bits(2)
return conversion
print("conversion de ", nombre_octets, " octet(s) en bit(s) : ", conve
def convertit_en_bits(nombre_octets):
# 1 octet = 8 bits
conversion = nombre_octets * 8
```

File "<basthon-input-1-f781643ab199>", line 6

```
conversion = nombre_octets * 8
^^^^^^^^^^
```

IndentationError: expected an indented block after function definition on line 4

Les instructions conditionnelles

Une **instruction conditionnelle**, ou instruction de test, permet de *faire des choix* en fonction de la valeur d'une *condition*. On parle souvent d'une instruction *si-alors*, ou *if-else* en anglais.

Une **condition** est une instruction qui est soit vraie, soit fausse. On dit qu'il s'agit d'un *booléen*.

Syntaxe d'une instruction conditionnelle

Pour écrire des instructions conditionnelles en Python, il faut utiliser la syntaxe suivante :

```
if condition1:
    bloc_instructions_1
elif condition2:
    bloc_instructions_2
else:
    bloc_instructions_3
```

Les mots-clés `if`, `elif` (contraction de *else if*) et `else` sont les traductions respectives de "si", "sinon si" et "sinon".



Exercice 2 : instructions conditionnelles



Question 1 : Ecris une fonction qui `est_positif(n)` qui renvoie `True` si le nombre passé en argument est positif ou nul, et `False` sinon.

Entrée[2]: *# écrire la fonction est_positif(n)*

```
print("est_positif(0) : ", est_positif(0))
print("est_positif(1) : ", est_positif(1))
print("est_positif(-1) : ", est_positif(-1))
```

Traceback (most recent call last):

```
File "<basthon-input-2-ec40f03aa51c>", line 5, in <module>
    print("est_positif(0) : ", est_positif(0))
                                ^^^^^^^^^^^^^
```

NameError: name 'est_positif' is not defined

 **Question 2** : Complète le code de la fonction `est_pair(n)` qui renvoie `True` si le nombre passé en paramètre est pair, et `False` sinon.

```
def est_pair(n):
    # un nombre est pair s'il est divisible par deux
    # un nombre est donc pair si le reste de la division par 2
    vaut zéro
    if (n%2 == 0):
        return ....
    .....
    .... False
```

Entrée[3]: *# complète*

```
def est_pair(n):
    # un nombre est pair s'il est divisible par deux
    # un nombre est donc pair si le reste de la division par 2 vaut zéro
    if (n%2 == 0):
        return ....
    .....
    .... False

print("est_pair(0) : ", est_pair(0))
print("est_pair(1) : ", est_pair(1))
print("est_pair(2) : ", est_pair(2))
print("est_pair(-4) : ", est_pair(-4))
print("est_pair(12) : ", est_pair(12))
print("est_pair(17) : ", est_pair(17))
```

```
File "<basthon-input-3-e8aabb85f835>", line 5
    return ....
    ^
```

SyntaxError: invalid syntax

 **Question 3** : Utilise l'appel des 2 fonctions précédentes pour écrire le code de la fonction `est_pair_positif(n)` qui renvoie `True` si le nombre passé en paramètre `n` est un nombre pair positif ou nul, et `False` sinon. Essaie d'écrire cette fonction en 5 lignes.

Entrée[6]: `# écrire la fonction est_pair_positif(n)`

```
print("est_pair_positif(0) : ", est_pair_positif(0))
print("est_pair_positif(1) : ", est_pair_positif(1))
print("est_pair_positif(2) : ", est_pair_positif(2))
print("est_pair_positif(-4) : ", est_pair_positif(-4))
print("est_pair_positif(12) : ", est_pair_positif(12))
print("est_pair_positif(17) : ", est_pair_positif(17))
print("est_pair_positif(19) : ", est_pair_positif(-19))
```

Traceback (most recent call last):

```
File "<basthon-input-6-3456b167f450>", line 6, in <module>
    print("est_pair_positif(0) : ", est_pair_positif(0))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "<basthon-input-4-ea5a576b6164>", line 2, in est_pair_positif
    f
```

```
    if est_positif(n):
    ^^^^^^^^^^^^^
```

NameError: name 'est_positif' is not defined. Did you mean: 'est_pair_positif'?

2. Les boucles

Souvent, dans un programme, il est nécessaire de répéter un certain nombre de fois une (ou des) instruction(s). Pour cela, on utilise ce qu'on appelle des **boucles**.

Les **boucles bornées**, qui nous intéressent ici sont utilisées **si on connaît à l'avance le nombre de répétitions à effectuer**.

Ces boucles bornées s'appellent également **boucles pour** ou **boucles for**.



Il existe également les **boucles non bornées** `while` qui sont utilisées dans le cas où **on ne connaît pas à l'avance le nombre de répétitions à effectuer**.

Exemple concret



Le nombre d'utilisateurs d'un nouveau réseau social est égal à 500 000 en janvier 2020. Ce nombre augmente de 5 % par mois, donc est multiplié par 1.05 chaque mois. Si on veut connaître le nombre d'utilisateurs 10 mois plus tard, il faut effectuer 10 fois de suite le même calcul (une multiplication par 1.05).

Au lieu d'écrire 10 lignes identiques il est préférable d'écrire une seule fois cette ligne et d'indiquer de l'exécuter 10 fois : pour cela, on utilise une boucle bornée.

En Python, au lieu d'écrire le programme

```
Entrée[33]: nb_utilisateurs = 500000
nb_utilisateurs = nb_utilisateurs * 1.05
nb_utilisateurs = nb_utilisateurs * 1.05
nb_utilisateurs = nb_utilisateurs * 1.05
nb_utilisateurs = nb_utilisateurs * 1.05
nb_utilisateurs = nb_utilisateurs * 1.05
nb_utilisateurs = nb_utilisateurs * 1.05
nb_utilisateurs = nb_utilisateurs * 1.05
nb_utilisateurs = nb_utilisateurs * 1.05
nb_utilisateurs = nb_utilisateurs * 1.05
nb_utilisateurs = nb_utilisateurs * 1.05

print(nb_utilisateurs)  # pour voir la valeur finale
```

814447.3133887209

on écrirait celui-ci :

```
Entrée[34]: nb_utilisateurs = 500000
for indice in range(10):
    nb_utilisateurs = nb_utilisateurs * 1.05

print(nb_utilisateurs)  # pour voir la valeur finale
```

814447.3133887209

Pour écrire une boucle bornée en Python on utilise le mot clé `for`. Les boucles bornées sont par conséquent très souvent appelées *boucles for* (ou *boucles pour* en français).

Le mot clé `for` peut s'utiliser de deux manières :

- avec la fonction `range()` : pour itérer sur une liste d'entiers
- seulement avec le mot-clé `in` : pour itérer sur les éléments de certaines "variables"

mais nous nous concentrerons dans cette séance uniquement sur la première possibilité avec `range()`

La fonction `range()`

Avant d'écrire des boucles `for`, il est important de comprendre le rôle de la fonction `range()`. Cette fonction permet de générer une liste d'entiers. On distingue trois cas de figure (les deux premiers sont les plus importants) :

- `range(n)` : permet de générer n entiers, de 0 (inclus) à n exclu, c'est-à-dire : 0, 1, 2, 3, ..., $n-1$
- `range(n, m)` : permet de générer $m-n$ entiers, de n (inclus) à m exclu, c'est-à-dire : $n, n+1, n+2, \dots, m-1$
- `range(n, m, p)` : permet de générer les entiers de n (inclus) à m exclus mais avec un pas égal à p , c'est-à-dire : $n, n+p, n+2p, \dots$

Exemples :

- `range(10)` génère la liste d'entiers : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 (il y en a 10 en tout, de 0 inclus à 10 exclu).
- `range(2, 10)` génère la liste d'entiers : 2, 3, 4, 5, 6, 7, 8, 9 (de 2 inclus à 10 exclu).
- `range(2, 10, 3)` génère la liste d'entiers : 2, 5, 8 (de 2 inclus à 10 exclu mais en allant de 3 en 3).

On peut visualiser la liste d'entiers générés en utilisant `list()`, comme ci-dessous :

```
Entrée[36]: list(range(10))
```

```
Sortie[36]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
Entrée[37]: list(range(2, 10))
```

```
Sortie[37]: [2, 3, 4, 5, 6, 7, 8, 9]
```

```
Entrée[38]: list(range(2, 10, 3))
```

```
Sortie[38]: [2, 5, 8]
```



Exercice 1

Indiquez dans chaque cas la liste d'entiers générée par :

1. `range(300)`
2. `range(1, 13)`
3. `range(50, 100)`
4. `range(3, 19, 2)`

Votre réponse :

- 1.
- 2.
- 3.
- 4.

N'hésitez pas à vérifier en testant votre code ci-dessous.

```
Entrée[40]: # pour tester
```

Boucles bornées avec la fonction `range()`

La syntaxe la plus simple d'une boucle `for` utilisant la fonction `range()` est la suivante.

```
for indice in range(n):  
    bloc_instructions
```

Et avec les deux autres cas d'utilisation de la fonction `range()` cela donne

```
for indice in range(n, m):  
    bloc_instructions
```

et

```
for indice in range(n, m, p):  
    bloc_instructions
```

Remarques :

- Il ne faut pas oublier le `:` à la fin de la ligne avec la boucle `for`
- La fonction `range` permet de générer une liste d'entiers comme vu précédemment.
- La variable notée `indice` prendra successivement (à chaque tour de boucle) les valeurs des entiers générés par la fonction `range`.
- Le bloc d'instructions à répéter doit être indenté (d'une tabulation) et sera donc ici exécuté autant de fois qu'il y a d'entiers générés par la fonction `range`.

Si on reprend le code de l'exemple concret, l'instruction `range(10)` va générer une liste de 10 entiers : 0, 1, 2, ..., 9. La variable `indice` va prendre successivement ces dix valeurs et donc le bloc d'instructions dans la boucle `for` sera exécuté 10 fois. On peut visualiser cela :

```
Entrée[44]: from tutor import tutor # utile pour exécuter pas à pas le programme  
  
nb_utilisateurs = 500000  
for indice in range(10):  
    nb_utilisateurs = nb_utilisateurs * 1.05  
  
tutor() # pour lancer la fenêtre interactive
```

Exercice 2 : les itérations (les répétitions)

On considère le programme Python suivant.

```
a = 2
for i in range(4):
    a = a + 1
a = 2 * a
```

 **Question 1** : Quel est le bloc d'instructions répété dans la boucle for ? Combien de fois est-il répété ?

Votre réponse :

 **Question 2** : Quelle est la valeur finale de la variable `a` ?

Votre réponse :

 **Question 3** : Recopiez ce programme dans la cellule ci-dessous en ajoutant la ligne permettant d'afficher `a` à la fin. Exécutez-le et vérifiez votre réponse à la question précédente.

Entrée[7]: *# à vous de coder !*

Exercice 3 : code-puzzle d'une fonction avec une boucle bornée for

 **Question 1** : Voici le code puzzle d'une fonction qui affiche les nombres de 0 à 20. Remets les lignes de code dans l'ordre et indente les lignes correctement. Puis exécute le code pour voir si tu as réussi.

```
affiche_nombres(0,20)
print(i)
def affiche_nombres(depart,fin):
    for i in range(fin+1):
```

Entrée[8]: *# à vous de coder !*

```
affiche_nombres(0,20)
print(i)
def affiche_nombres(depart,fin):
for i in range(fin+1):
```

```
File "<basthon-input-8-58379b762012>", line 6
    for i in range(fin+1):
    ^^^
```

IndentationError: expected an indented block after function definition on line 5



Exercice 4 : écrire une fonction avec une boucle bornée for



Question 1 : Ecris une fonction `nombres_pairs(debut,fin)` qui affiche les nombres entre 2 et 20. Tu peux utiliser les fonctions de l'introduction.

Lien vers ces fonctions [Syntaxe d'une fonction Python](#) pour t'aider à répondre.

Entrée[9]:

```
def est_pair(n):
    if (n%2 == 0):
        return True
    else:
        return False
```

à toi de coder la fonction nombres_pairs(debut,fin):

```
# appel de la fonction
nombres_pairs(2,20)
```

Traceback (most recent call last):

```
File "<basthon-input-9-5745503c6a87>", line 12, in <module>
    nombres_pairs(2,20)
    ^^^^^^^^^^^^^^^
```

NameError: name 'nombres_pairs' is not defined



Question 2 : Ecris une fonction `nombres_pairs_while(debut,fin)` qui fait comme la fonction précédente mais avec une boucle `while`.

```
Entrée[4]: def est_pair(n):  
            if (n%2 == 0):  
                return True  
            else:  
                return False  
  
            # à toi de coder la fonction  
            # n'oublie pas d'incrémenter ton compteur dans la boucle while pour év  
            # Sauvegarde ton notebook avant d'exécuter ta fonction.  
  
            def nombres_pairs_while(debut,fin):  
  
  
  
  
  
  
  
  
  
            # appel de la fonction  
            nombres_pairs_while(2,20)  
  
2  
4  
6  
8  
10  
12  
14  
16  
18  
20
```

3. Les bibliothèques

Certaines fonctions Python sont déjà écrites et regroupées dans des modules. Elles nécessitent d'être importées pour être utilisées. Pour utiliser toutes les fonctions du module `nom_module`, on ajoute en début de programme l'instruction : `from nom_module import *`

Exemples de modules :

- **math** (pour les fonctions sqrt, pow(x,y), round, cos(x), sin(x), log(x), exp(x), la constante pi, la constante e...)
- **random** (fonctions randint(x,y) pour avoir un nomnbre entier aléatoire entre x et y, random() qui retourne un nombre réel entre 0 et 1)
- **turtle** (fonctions pour dessiner : forward, right, left, backward...)

```
from math import *  
from random import *  
from turtle import *
```



Exercice 1 : utiliser le module random



Question 1 : le lancer de dé est un jeu de hasard cours duquel on peut obtenir des entiers entre 1 et 6. On s'intéresse à la fonction `lance_de(1,6)` servant à afficher un tirage aléatoire de dé. **Exécute la cellule plusieurs fois et observe les résultats.**

```
Entrée[15]: from random import randint  
  
def lance_de(depart,fin):  
    lancer = randint(depart,fin)  
    print("tirage : ",lancer)  
    return lancer  
  
lance_de(1,6)  
lance_de(1,6)  
lance_de(1,6)  
lance_de(1,6)  
lance_de(1,6)  
lance_de(1,6)
```

```
tirage : 1  
tirage : 5  
tirage : 4  
tirage : 1  
tirage : 3  
tirage : 2
```

Sortie[15]: 2



Question 2 : Que se passe-t-il et pourquoi ?

Entrée[]: Réponse :

Exercice 2 : utiliser le module math et corriger des erreurs

 **Question 1** : Corrige les erreurs de la fonction ci-dessous afin qu'elle retourne l'aire d'un cercle de rayon 10 cm (soit environ 314 cm).

```
constante_pi = 3,14
def aire-cercle(rayon)
    aire = constante_pi * r * 2
    return aire

print("aire du cercle de rayon : ",aire_cercle(10))
```

Rappels :

- Périmètre du disque = $2 \times \pi \times$ rayon du disque = $2 \times \pi \times r$
- Aire du disque = $\pi \times$ rayon du disque $\times$ rayon du disque = $\pi \times r \times r$ où π est un nombre à peu près égal à 3,14.

Entrée[28]:

```
constante_pi = 3,14
def aire-cercle(rayon)
    aire = constante_pi * r * 2
    return aire

print("aire du cercle de rayon : ",aire_cercle(10))
```

```
File "<basthon-input-28-681f723b92c6>", line 2
    def aire-cercle(rayon)
        ^
```

SyntaxError: expected '('

Activité 3 : Dessiner avec la tortue

pen down



Crédit : [Cormullion \(https://commons.wikimedia.org/wiki/File:Turtle-animation.gif\)](https://commons.wikimedia.org/wiki/File:Turtle-animation.gif), [CC BY-SA 4.0 \(https://creativecommons.org/licenses/by-sa/4.0\)](https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons.

Dans Scratch, on trouve quelques *blocs* de base, pour avancer ou faire tourner le chat :



Cela permettait, entre autres, de faire dessiner le chat (ou le lutin).

En Python, on peut utiliser le module `turtle` (= tortue en français) pour dessiner. Avec ce module, les équivalents des blocs de base précédents sont les **fonctions** `forward`, `right` et `left`, qui permettent de faire avancer ou tourner une tortue :

- `forward(x)` pour faire avancer de `x` pixels
- `right(a)` pour faire tourner de `a` degrés vers la droite
- `left(a)` pour faire tourner de `a` degrés vers la gauche

Pour dessiner, on appelle ces fonctions les unes à la suite des autres avec les paramètres souhaités (c'est l'équivalent d'imbriquer les blocs les uns à la suite des autres avec Scratch)

Par exemple :

```
forward(100)
right(90)
forward(50)
left(45)
forward(200)
```

Pour utiliser ces fonctions il faut dire en première ligne à Python que l'on veut charger le module `turtle`. Cela se fait avec l'instruction

```
from turtle import *
```

Exécutez les instructions ci-dessous pour voir un premier dessin !

```
Entrée[50]: from turtle import *  # pour charger le module turtle

forward(100)
right(90)
forward(50)
left(45)
forward(200)

done()  # pour demander l'affichage du tracé
```



La dernière instruction, `done()`, permet de demander l'affichage du tracé. Si on l'oublie, il ne se passe rien.

 **Question 1** : Combien les fonctions `forward`, `right` et `left` possèdent-elles de paramètres ? Expliquer rapidement.

Votre réponse :

 **Question 2** : Complétez les instructions suivantes pour dessiner un carré, de 50 pixels de côté. Puis exécutez les instructions pour vérifier (et corriger si nécessaire).

```
from turtle import *

forward(...)
...
...
...
...
...
...
...

done()
```



```
Entrée[ ]: from turtle import *

forward(...)
...
...
...
...
...
...
...

done()
```

 **Question 3** : Écrivez les instructions permettant de dessiner un carré, de 100 pixels de côté, puis les exécutez.

```
Entrée[53]: # à vous de jouer !
from turtle import *

done()
```



Normalement, vous avez du écrire quasiment les mêmes instructions aux questions 2 et 3. Ce n'est pas forcément efficace. Vous allez voir qu'il est préférable de définir une fonction permettant de dessiner un carré puis de l'appeler à chaque fois que l'on veut dessiner un carré.

 **Question 4** : On veut écrire une fonction `carre` qui possède un paramètre `taille` qui correspond à la taille des côtés du carré dessiné.

Exemples : L'appel `carre(50)` devra dessiner un carré de côté 50 pixels, l'appel `carre(80)` devra dessiner un carré de côté 80 pixels.

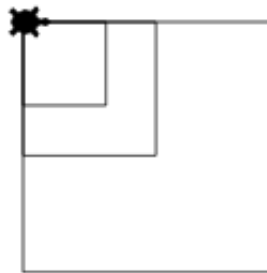
```
Entrée[ ]: def carre(taille):
    forward(...)
    right(...)
    ... # à poursuivre
```

Pensez à bien exécuter la cellule ci-dessus pour mémoriser la fonction `carre` .

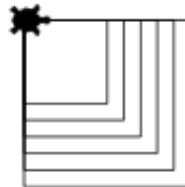
On peut maintenant appeler cette fonction pour tracer d'affilée trois carrés de tailles différentes :

```
Entrée[57]: carre(80)  
            carre(50)  
            carre(150)  
  
            done( )
```

Ces instructions doivent produire le dessin ci-dessous, si ce n'est pas le cas il faut corriger le code de votre fonction `carre` .



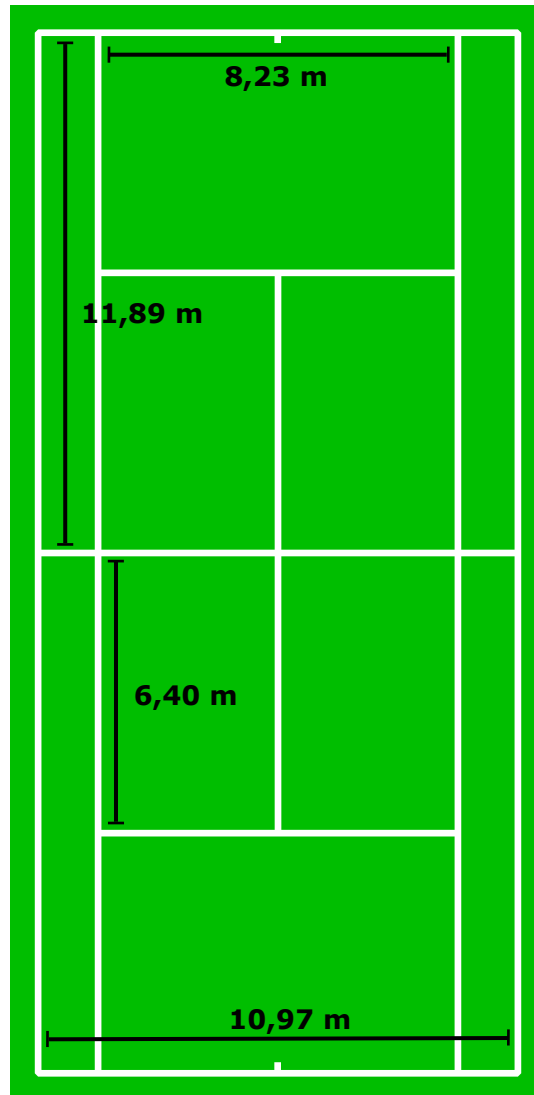
 **Question 5** : On veut obtenir la figure ci-dessous. Écrivez les bonnes instructions. *Ne pas oublier de terminer par `done( )` !*



```
Entrée[61]: # à vous de jouer !
```

💪 Défi !

Créez une fonction `rectangle` qui permet de dessiner un rectangle qui possède la longueur et la largeur voulue, puis utilisez cette fonction pour dessiner un court de tennis dont on rappelle les dimensions ci-dessous.



Entrée[76]: *# à vous de jouer !*

Bilan

- Une **fonction** permet de regrouper une série d'instructions.
- Une fonction possède un **nom**, éventuellement des **paramètres** qui sont utilisés dans le **bloc d'instructions** de la fonction. La fonction **renvoie** généralement une ou plusieurs valeurs.
- On peut faire **appel** à une fonction pour exécuter les instructions qu'elle contient et connaître la valeur renvoyée par celle-ci. On l'appelle par son nom en précisant les valeurs des paramètres souhaités entre parenthèses.
- En Python :
 - on utilise le mot clé `def` pour annoncer que l'on va écrire une fonction
 - suivi du nom de la fonction puis de ses paramètres écrits entre parenthèses
 - il ne faut pas oublier le `:` à la fin de la première ligne
 - le bloc d'instructions d'une fonction doit être *indenté*
 - c'est le mot clé `return` qui permet de renvoyer une ou plusieurs valeurs

```
def nom_fonction(parametre1, parametre2, etc.):  
    instruction1  
    instruction2  
    ...  
    return valeur_renvoyee_1, valeur_renvoyee_2, etc.
```

- L'un des intérêts d'une fonction est de pouvoir regrouper des instructions susceptibles d'être utilisées à plusieurs reprises. Cela permet donc d'alléger le code puisqu'il suffit ensuite d'appeler la fonction sans avoir à recopier toutes ses instructions à chaque fois.
- Il y a d'autres raisons qui font que l'on utilise beaucoup les fonctions en programmation mais nous ne les aborderons pas.

Références :

- Les travaux d'Alexandre André sur le module `turtle` : De Scratch à Python : le chat et la tortue (<https://capytale2.ac-paris.fr/basthon/notebook/?id=32146>) et Tortue : fonctions, frises, feu d'artifice (<https://capytale2.ac-paris.fr/basthon/notebook/?id=33841>) pour les idées de l'activité sur Turtle.
- Les enseignants de SNT du lycée Emmanuel Mounier, ANGERS



Entrée[]: